

ROS 2 Publisher / Subscriber

rclpy · Topic · QoS · Message Types

cchyun@gmail.com

Part 1. Publisher 심화

- ROS 2에서 토픽(Topic)은 노드 간 연속적인 데이터 스트림을 주고받는 가장 기본적인 통신 방식
 - Publisher (발행자)
 - 데이터를 특정 토픽 채널로 지속적으로 보내는 쪽
 - Subscriber (구독자)
 - 특정 토픽 채널을 감시하며 데이터가 도착하면 받는 쪽
 - 핵심 원칙
 - Publisher와 Subscriber의 토픽 이름이 정확히 일치해야 통신 가능
 - 1:1, 1:N, N:1, N:N 구조 모두 지원 — 느슨한 결합(Loose Coupling)

클래스 기반 노드를 만드는 이유

- ROS 2는 `rclpy.node.Node` 클래스를 상속받아 노드를 설계하는 것이 표준
 - 객체지향 설계의 이점
 - 노드의 상태(변수)와 기능(메서드)를 하나의 클래스에 캡슐화
 - 코드의 조직화·재사용성 향상
 - 클래스 기반 노드 구조
 - `class MyNode(Node):` — `Node` 클래스를 상속
 - `super().__init__('node_name')` — 부모 생성자 호출, 노드 이름 설정
 - `__init__()` 내부에서 `Publisher·Timer` 초기화
 - 실수 방지 포인트
 - `super().__init__()` 호출을 빠뜨리면 노드가 초기화되지 않아 오류 발생

create_publisher() 파라미터 상세

- `self.create_publisher()` — 데이터를 보낼 통로를 개설하는 메서드
 - ① `msg_type` — 메시지 타입
 - 통신에 사용할 메시지 클래스
 - 예: `std_msgs.msg.String`, `sensor_msgs.msg.LaserScan`
 - ② `topic_name` — 토픽 채널 이름
 - 데이터를 보낼 채널의 고유 이름
 - Publisher와 Subscriber가 동일한 이름을 사용해야 연결됨
 - ③ `qos_profile` — 서비스 품질(QoS) 설정
 - `QoSProfile`로 신뢰성·내구성 등을 세밀하게 지정 가능
 - 정수를 입력하면 `history depth`(큐 사이즈)로 인식

```
qos = QoSProfile(  
    reliability=ReliabilityPolicy.RELIABLE,  
    durability=DurabilityPolicy.VOLATILE,  
    depth=10  
)  
self.publisher_ = self.create_publisher(String, 'my_topic', qos)
```

create_timer() — 타이머 콜백 동작 원리

- `self.create_timer(timer_period, callback)` — 지정 주기마다 함수를 반복 실행

- 동작 원리

- `timer_period` (초 단위) 마다 이벤트를 발생시켜 등록된 콜백 함수를 호출
- 콜백 내부에서 메시지를 생성하고 `publish()` 호출

```
self.timer = self.create_timer(0.5, self.timer_callback) # 0.5초 = 2Hz
```

- 콜백 충돌 시 동작 (단일 스레드 환경)

- 기본적으로 한 번에 하나의 콜백만 실행
- 콜백 실행 시간이 타이머 주기보다 길어지면 → 다음 실행이 지연됨

- 주의: 콜백 내부에서 `time.sleep()` 을 길게 사용하면 노드 전체가 멈춤

- 노드가 "살아있게" 하고 콜백을 처리하기 위해 Executor(실행기)가 필요
 - `rclpy.spin(node)` — 무한 루프
 - 노드를 종료할 때까지 모든 콜백을 계속 처리
 - 가장 일반적인 사용 패턴
 - `rclpy.spin_once(node)` — 단발성 처리
 - 대기 중인 콜백을 단 한 번만 처리하고 즉시 반환
 - 특수한 루프 제어가 필요할 때 사용
 - `MultiThreadedExecutor` — 병렬 처리
 - 여러 스레드를 사용하여 콜백을 동시에 병렬로 처리
 - 처리량이 많거나 응답성이 중요한 시스템에서 사용

```
from rclpy.executors import MultiThreadedExecutor
executor = MultiThreadedExecutor()
executor.add_node(node)
executor.spin()
```

• QoS(Quality of Service) — 네트워크 환경에 맞춰 통신 특성을 조절하는 설정

• Reliability (신뢰성) 정책

설정	동작	적합한 데이터
RELIABLE	손실 시 재전송 보장	명령, 파라미터 설정
BEST_EFFORT	손실 허용, 지연 최소화	LiDAR, 카메라 고주파 센서

• Durability (내구성) 정책

설정	동작	특징
VOLATILE	접속 이후 메시지만 수신	일반적인 스트리밍
TRANSIENT_LOCAL	이전 메시지도 신규 구독자에게 전달	초기 상태 동기화가 필요한 경우

QoS — 기본값 (Default QoS Profile)

- QoS를 별도로 지정하지 않거나 정수를 넘기면 아래 기본값이 적용됨

필드	기본값	설명
<code>reliability</code>	<code>RELIABLE</code>	메시지 손실 시 재전송 보장
<code>durability</code>	<code>VOLATILE</code>	접속 이전 메시지는 수신하지 않음
<code>history</code>	<code>KEEP_LAST</code>	최신 메시지 N개만 큐에 보관
<code>depth</code>	<code>10</code>	큐에 보관할 메시지 수
<code>liveliness</code>	<code>AUTOMATIC</code>	노드가 살아있음을 자동으로 알림
<code>deadline</code> / <code>lifespan</code>	무제한	메시지 유효 기간 제한 없음

- 정수를 넘기면 `depth` 만 지정, 나머지는 기본값 적용

- 데이터 특성에 따른 QoS 설정 선택 (1/2)
 - 센서 데이터 (LiDAR, 카메라)
 - `BEST_EFFORT` + `VOLATILE`
 - 최신성이 중요 → 한두 개 누락되어도 문제없음
 - 로봇 명령 / 상태 정보
 - `RELIABLE` + `TRANSIENT_LOCAL`
 - 누락되면 안 되며, 초기 상태 확인이 필요

- 데이터 특성에 따른 QoS 설정 선택 (2/2)

- QoS Mismatch 주의

- Publisher가 `BEST_EFFORT` ↔ Subscriber가 `RELIABLE` 요청
- → 연결 자체가 이루어지지 않음

```
from rclpy.qos import QoSProfile, ReliabilityPolicy, DurabilityPolicy
qos = QoSProfile(
    reliability=ReliabilityPolicy.BEST_EFFORT,
    durability=DurabilityPolicy.VOLATILE,
    depth=5
)
```

여러 토픽을 퍼블리시하는 노드 설계

- 하나의 노드에서 여러 개의 Publisher를 운용하는 것이 가능
 - 구조 설계
 - 생성자(`__init__`)에서 여러 `create_publisher()` 호출
 - 하나의 타이머 콜백 안에서 순차적으로 `publish()` 호출
 - 또는 각각 다른 타이머를 사용하여 독립적으로 발행
 - 장점
 - 관련 있는 데이터를 하나의 노드에서 관리 → 데이터 동기화가 쉬움

```
def __init__(self):
    super().__init__('multi_publisher_node')
    self.pub_temp = self.create_publisher(Float32, 'temperature', 10)
    self.pub_humidity = self.create_publisher(Float32, 'humidity', 10)
    self.timer = self.create_timer(1.0, self.publish_all)

def publish_all(self):
    self.pub_temp.publish(...)
    self.pub_humidity.publish(...)
```

- 초보자가 Publisher 구현 시 빈번하게 겪는 오류 패턴
 - ① `spin()` 누락
 - Publisher만 만들고 `rclpy.spin(node)` 를 호출하지 않으면
 - → 노드가 즉시 종료되어 아무 일도 일어나지 않음
 - ② QoS Mismatch
 - Publisher와 Subscriber의 QoS 설정이 호환되지 않으면
 - → `ros2 topic echo` 에서 아무것도 수신되지 않음
 - ③ 콜백 내 블로킹 코드
 - 콜백 함수 안에서 `time.sleep()` 을 길게 사용
 - → 다른 콜백 처리가 모두 멈춰 노드가 먹통이 됨

(실습) LiDAR 거리 데이터 Publisher

```
import rclpy
from rclpy.node import Node
from std_msgs.msg import Float64
import random

class LidarPublisher(Node):
    def __init__(self):
        super().__init__('lidar_publisher_node')
        qos_profile = rclpy.qos.QoSProfile(
            reliability=rclpy.qos.ReliabilityPolicy.BEST_EFFORT,
            durability=rclpy.qos.DurabilityPolicy.VOLATILE,
            depth=5
        )
        self.publisher_ = self.create_publisher(Float64, 'scan_distance', qos_profile)
        self.timer = self.create_timer(0.1, self.publish_lidar_data) # 10Hz
        self.get_logger().info('LiDAR Publisher가 시작되었습니다.')

    def publish_lidar_data(self):
        msg = Float64()
        msg.data = random.uniform(1.0, 5.0) # 가상 LiDAR 데이터 1.0~5.0m
        self.publisher_.publish(msg)
        self.get_logger().info(f'Published Distance: {msg.data:.2f}m')

def main(args=None):
    rclpy.init(args=args)
    node = LidarPublisher()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        rclpy.shutdown()
```

Part 2. Subscriber 심화 + 메시지 타입

create_subscription() 파라미터 상세

- `self.create_subscription()` — 지정 토픽을 감시하고 데이터 수신 시 콜백 실행 (1/2)
 - ① `msg_type` — 메시지 타입
 - 구독하려는 메시지의 클래스 타입
 - 발행 측과 반드시 일치해야 데이터를 올바르게 해석 가능
 - ② `topic_name` — 토픽 이름
 - Publisher와 동일한 이름 사용 필수
 - ③ `callback` — 수신 콜백 함수
 - 메시지가 도착할 때마다 자동으로 호출
 - 타이머 없이 데이터 도착 즉시 실행

create_subscription() 파라미터 상세

- `self.create_subscription()` — 지정 토픽을 감시하고 데이터 수신 시 콜백 실행 (2/2)
 - ④ `qos_profile` — QoS 설정
 - Publisher의 QoS와 호환 가능해야 연결 성립

```
self.subscription = self.create_subscription(  
    LaserScan, '/scan', self.scan_callback, qos_profile)
```

- Subscriber 콜백 함수는 빠르고 가볍게 설계하는 것이 핵심
 - 무거운 연산을 피해야 하는 이유
 - ROS 2는 비차단(Non-blocking) 동작이 중요
 - 콜백에서 복잡한 계산이나 대기가 발생하면
 - → Executor가 다른 메시지/타이머 이벤트를 처리하지 못해 응답성 저하
 - 권장 패턴: 멤버 변수 저장
 - 콜백에서는 수신 데이터를 멤버 변수에만 저장
 - 실제 연산은 별도 타이머 콜백이나 메인 루프에서 처리

```
def scan_callback(self, msg):  
    self.latest_scan = msg  # 저장만 하고 빠르게 반환  
  
def process_timer_callback(self):  
    if self.latest_scan:  
        # 여기서 무거운 처리 수행  
        ...
```

- **std_msgs** — 기본 데이터 타입
 - **String** — 문자열 상태 알림, 간단한 명령
 - **Float32 / Float64** — 실수형 센서값
 - **Int32 / Int64** — 정수형 카운트
 - **Bool** — ON/OFF 상태 플래그
- **sensor_msgs** — 센서 데이터 관련 타입

타입	설명	주요 필드
LaserScan	LiDAR 거리 배열	ranges , 각도 범위
Image	카메라 영상	픽셀 데이터, 인코딩 정보
Imu	가속도·각속도·방향	자세 제어 활용
PointCloud2	3D 점 구름	정밀 환경 인식

- `geometry_msgs` — 기하학적 형상과 운동 관련 타입

타입	설명
<code>Twist</code>	선속도 + 각속도 → <code>cmd_vel</code> 표준
<code>Pose / PoseStamped</code>	위치(Position) + 방향(Orientation)
<code>Vector3</code>	x, y, z 방향 벡터

- `nav_msgs` — 자율 주행 및 지도 관련 타입

타입	설명
<code>Odometry</code>	로봇 추정 위치 + 속도
<code>OccupancyGrid</code>	2D 격자 지도, 각 격자 점유 확률
<code>Path</code>	<code>PoseStamped</code> 배열 형태의 경로

Header 메시지의 역할

- `std_msgs/msg/Header` — 많은 메시지 타입이 포함하는 맥락 정보
 - `stamp` (타임스탬프)
 - 메시지가 생성된 정확한 시점을 기록
 - 여러 센서 데이터의 동기화 및 과거 상태 추적에 필수
 - `frame_id` (좌표계 이름)
 - 데이터가 측정된 기준 좌표계의 이름
 - 예: `base_link`, `laser_frame`, `camera_frame`
 - 여러 센서가 장착된 로봇에서 각 데이터의 기준 위치를 구분
 - 활용 예시
 - `LaserScan.header.frame_id = "laser_frame"` → 레이저 센서 기준 좌표
 - `PoseStamped.header.stamp` → 해당 위치 정보가 언제 측정되었는지 표시

- 하나의 클래스 기반 노드에서 여러 Subscriber를 생성하는 것이 가능
 - 설계 원칙
 - 각 Subscriber마다 **전용 콜백 함수**를 지정
 - 수신된 데이터를 각각 **멤버 변수에 업데이트**하여 관리
 - 데이터 동기화 주의사항
 - 서로 다른 토픽의 데이터 도착 시점이 다를 수 있음
 - 엄격한 동기화가 필요하면 **별도 동기화 로직 추가** 필요

```
def __init__(self):
    super().__init__('multi_subscriber_node')
    self.sub_scan = self.create_subscription(
        LaserScan, '/scan', self.scan_callback, 10)
    self.sub_imu = self.create_subscription(
        Imu, '/imu', self.imu_callback, 10)
    self.latest_scan = None
    self.latest_imu = None
```

- Subscriber 구현 시 빈번하게 겪는 오류 패턴

- ① QoS Mismatch

- 발행자 `BEST_EFFORT` ↔ 구독자 `RELIABLE` 요청
- 통신 자체가 이루어지지 않음
- `ros2 topic info /topic_name --verbose` 로 QoS 확인 가능

- ② 동기적 데드락(Deadlock)

- 콜백 내부에서 다른 서비스를 동기적으로 호출
- 노드가 멈추는 현상 발생
- 해결: 비동기 호출 `call_async()` 사용

- ③ 토픽 이름 / 타입 불일치

- 오타나 잘못된 메시지 클래스 사용
- `rqt_graph` 에서 연결된 것처럼 보여도 실제 데이터 미전달
- `ros2 topic echo /topic_name` 으로 반드시 확인

(실습) LaserScan 구독 및 최단 거리 출력

```
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import LaserScan
from rclpy.qos import QoSProfile, ReliabilityPolicy

class ObstacleDetector(Node):
    def __init__(self):
        super().__init__('obstacle_detector')
        qos_profile = QoSProfile(
            reliability=ReliabilityPolicy.BEST_EFFORT,
            depth=10
        )
        self.subscription = self.create_subscription(
            LaserScan,
            '/scan',
            self.scan_callback,
            qos_profile)
        self.get_logger().info('LaserScan 구독을 시작합니다.')

    def scan_callback(self, msg):
        # 유효한 거리 데이터 중 최소값 (0.0 또는 inf 제외)
        valid_ranges = [r for r in msg.ranges if msg.range_min < r < msg.range_max]
        if valid_ranges:
            min_distance = min(valid_ranges)
            self.get_logger().info(f'최단 장애물 거리: {min_distance:.2f} m')
        else:
            self.get_logger().warn('유효한 거리 데이터 없음')

def main(args=None):
    rclpy.init(args=args)
    node = ObstacleDetector()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        rclpy.shutdown()
```


Part 3. 퀴즈 & 복습

퀴즈 1 — 코드 읽기 (동작 예측)

- 다음 코드를 보고 동작을 예측해 보세요

- Q1. `self.create_timer(1.0, self.timer_callback)` 이 포함된 노드에서 `timer_callback` 함수는 얼마나 자주 실행되나요?
- Q2. `self.create_subscription(String, 'sensor_data', self.listener_callback, 10)` 이 노드가 데이터를 수신하는 시점은 언제인가요?
- Q3. `rclpy.spin(node)` 를 코드 마지막에 호출하는 이유는 무엇인가요?
- Q4. 동일한 Publisher 노드를 두 개 실행할 때 토픽 이름이 겹치지 않게 하려면 어떤 설정을 사용해야 하나요?
- Q5. `self.declare_parameter('threshold', 0.5)` 코드의 목적은 무엇인가요?

퀴즈 2 — QoS 선택 (상황 맞춤형)

- 로봇 시스템 통신 환경에 가장 적합한 QoS 설정을 골라보세요
 - 상황 A. 무선 네트워크에서 **카메라 센서 데이터**를 고속 전송
 - 한두 개 누락되어도 최신 데이터를 받는 것이 중요할 때
 - → 적합한 **Reliability** 설정은?
 - 상황 B. 비상 정지 명령처럼 반드시 전달되어야 하는 데이터
 - → 사용해야 하는 **Reliability** 설정은?
 - 상황 C. 노드가 늦게 켜지더라도 **마지막 로봇 상태 정보**를 즉시 받고 싶을 때
 - → Publisher와 Subscriber가 공통으로 사용해야 하는 **Durability** 설정은?
 - 상황 D. Publisher는 **Best Effort**, Subscriber는 **Reliable** 요청
 - → 두 노드 사이의 통신은 어떻게 되나요?

퀴즈 3 — 메시지 타입 매칭

- 센서/기능에 적합한 메시지 타입을 연결해 보세요
 - A. 3차원 공간의 좌표 데이터(X, Y, Z)
 - B. 간단한 텍스트 상태 알림
 - C. 연속적으로 흐르는 센서 데이터 스트림 통신 방식
 - D. 로봇의 이동 속도 명령 (`cmd_vel`)
 - E. 로봇 주변 360° 거리 배열 데이터
- 보기:
 - `geometry_msgs/msg/Twist`
 - `std_msgs/msg/String`
 - `sensor_msgs/msg/LaserScan`
 - `geometry_msgs/msg/Point`
 - `Topic`



- 프로젝트 진행 시 다음 단계를 모두 수행했는지 점검하세요
 - 환경 설정
 - [] 새 터미널을 열 때마다 `source /opt/ros/humble/setup.bash` 실행
 - 빌드 & 의존성
 - [] `colcon build --packages-select [패키지명]` 으로 정상 빌드 확인
 - [] `package.xml` 에 `rclpy`, `std_msgs` 등 의존성 패키지 추가
 - 동작 확인
 - [] `ros2 topic echo /[토픽이름]` 으로 데이터 실시간 발행 확인
 - [] `ros2 topic info /[토픽이름] --verbose` 로 QoS 설정 확인
 - [] `rqt_graph` 로 노드-토픽 연결 관계 시각적으로 확인